Getting help
RTL
Layout
Breakpoints
Grid
Stacks
Forms
Components
Accordion
Alerts
Badges
Breadcrumbs
Button group
Buttons
Cards
Carousels
Close Button
Dropdowns
Figures
Images
List groups
Modals
Navbars
Navs and tabs

Examples
Outline buttons
Button tags
Sizes
Block buttons
Active state
Disabled state
Button loading state
Checkbox / Radio
API
Button
ToggleButtonGroup
ToggleButton

Buttons

Use Bootstrap's custom button styles for actions in forms, dialogs, and more with support for multiple sizes, states, and more.

Examples

Use any of the available button style types to quickly create a styled button. Just modify the `variant` prop.

RESULT

Primary Secondary Success Warning Danger Info Light Dark Link

LIVE EDITOR

```
import Button from 'react-bootstrap/Button';

function TypesExample() {
  return (
    <>
      <Button variant="primary">Primary</Button>
      <Button variant="secondary">Secondary</Button>
      <Button variant="success">Success</Button>
      <Button variant="warning">Warning</Button>
      <Button variant="danger">Danger</Button>
      <Button variant="info">Info</Button>
      <Button variant="light">Light</Button>
      <Button variant="dark">Dark</Button>
      <Button variant="link">Link</Button>
    </>
  );
}

export default TypesExample;
```

Outline buttons

For a lighter touch, Buttons also come in `outline-*` variants with no background color.

RESULT

Primary Secondary Success Warning Danger Info Light Dark

LIVE EDITOR

```
import Button from 'react-bootstrap/Button';

function OutlineTypesExample() {
  return (
    <>
      <Button variant="outline-primary">Primary</Button>
      <Button variant="outline-secondary">Secondary</Button>
      <Button variant="outline-success">Success</Button>
      <Button variant="outline-warning">Warning</Button>
      <Button variant="outline-danger">Danger</Button>
      <Button variant="outline-info">Info</Button>
      <Button variant="outline-light">Light</Button>
      <Button variant="outline-dark">Dark</Button>
    </>
  );
}

export default OutlineTypesExample;
```

Button tags

Normally `<Button>` components will render a HTML `<button>` element. However you can render whatever you'd like, adding a `href` prop will automatically render an `<a />` element. You can use the `as` prop to render whatever your heart desires. React Bootstrap will take care of the proper ARIA roles for you.

RESULT

Link Button Input Submit Reset

LIVE EDITOR

```
import Button from 'react-bootstrap/Button';

function TagTypesExample() {
  return (
    <>
      <Button href="#">Link</Button> <Button type="submit">Button</Button>
      <Button as="input" type="button" value="Input" />
      <Button as="input" type="submit" value="Submit" />
      <Button as="input" type="reset" value="Reset" />
    </>
  );
}

export default TagTypesExample;
```

Sizes

Fancy larger or smaller buttons? Add `size="lg"`, `size="sm"` for additional sizes.

RESULT

Large button Large button
Small button Small button

LIVE EDITOR

```
import Button from 'react-bootstrap/Button';

function SizesExample() {
  return (
    <>
      <div className="d-flex gap-2 mb-2">
        <Button variant="primary" size="lg">
          Large button
        </Button>
        <Button variant="secondary" size="lg">
          Large button
        </Button>
      </div>
      <div className="d-flex gap-2">
        <Button variant="primary" size="sm">
          Small button
        </Button>
        <Button variant="secondary" size="sm">
          Small button
        </Button>
      </div>
    </>
  );
}

export default SizesExample;
```

Block buttons

Create responsive stacks of full-width, "block buttons" like those in Bootstrap 4 with a mix of our display and gap utilities.

RESULT

Block level button
Block level button

LIVE EDITOR

```
import Button from 'react-bootstrap/Button';

function BlockExample() {
  return (
    <div className="d-grid gap-2">
      <Button variant="primary" size="lg">
        Block level button
      </Button>
      <Button variant="secondary" size="lg">
        Block level button
      </Button>
    </div>
  );
}

export default BlockExample;
```

Active state

To set a button's active state simply set the component's `active` prop.

RESULT

Primary button Button

LIVE EDITOR

```
import Button from 'react-bootstrap/Button';

function ActiveExample() {
  return (
    <>
      <Button variant="primary" size="lg" active>
        Primary button
      </Button>
      <Button variant="secondary" size="lg" active>
        Button
      </Button>
    </>
  );
}

export default ActiveExample;
```

Disabled state

Make buttons look inactive by adding the `disabled` prop to.

RESULT

Primary button Button Link

LIVE EDITOR

```
import Button from 'react-bootstrap/Button';

function DisabledExample() {
  return (
    <>
      <Button variant="primary" size="lg" disabled>
        Primary button
      </Button>
      <Button variant="secondary" size="lg" disabled>
        Button
      </Button>
      <Button href="#" variant="secondary" size="lg" disabled>
        Link
      </Button>
    </>
  );
}

export default DisabledExample;
```

Watch out! `<a>` elements don't naturally support a `disabled` attribute. In browsers that support it this is handled with a `point-events: none` style but not all browsers support it yet.

React Bootstrap will prevent any `onClick` handlers from firing regardless of the rendered element.

Button loading state

When activating an asynchronous action from a button it is a good UX pattern to give the user feedback as to the loading state, this can easily be done by updating your `<Button />`s props from a state change like below.

RESULT

Click to load

LIVE EDITOR

```
import { useEffect, useState } from 'react';
import Button from 'react-bootstrap/Button';

function LoadingButton() {
  const [isLoading, setLoading] = useState(false);

  useEffect(() => {
    function simulateNetworkRequest() {
      return new Promise(resolve => {
        setTimeout(resolve, 2000);
      });
    }

    if (isLoading) {
      simulateNetworkRequest().then(() => {
        setLoading(false);
      });
    }
  }, [isLoading]);

  const handleClick = () => setLoading(true);

  return (
    <Button
      variant="primary"
      disabled={isLoading}
      onClick={!isLoading ? handleClick : null}
    >
      {isLoading ? 'Loading…' : 'Click to load'}
    </Button>
  );
}

export default LoadingButton;
```

Checkbox / Radio

Buttons can also be used to style `checkbox` and `radio` form elements. This is helpful when you want a toggle button that works neatly inside an HTML form.

RESULT

Checked
Active Radio Radio
Checked
Active Radio Radio

LIVE EDITOR

```
import { useState } from 'react';
import ButtonGroup from 'react-bootstrap/ButtonGroup';
import ToggleButton from 'react-bootstrap/ToggleButton';

function ToggleButtonExample() {
  const [checked, setChecked] = useState(false);
  const [radioValue, setRadioValue] = useState('1');

  const radios = [
    { name: 'Active', value: '1' },
    { name: 'Radio', value: '2' },
    { name: 'Radio', value: '3' },
  ];

  return (
    <>
      <ButtonGroup className="mb-2">
        <ToggleButton
          id="toggle-check"
          type="checkbox"
          variant="secondary"
          checked={checked}
          value="1"
          onChange={(e) => setChecked(e.currentTarget.checked)}
        >
          Checked
        </ToggleButton>
      </ButtonGroup>
      <br />
      <ButtonGroup className="mb-2">
        {radios.map((radio, idx) => (
          <ToggleButton
            key={idx}
            id={`radio-${idx}`}
            type="radio"
            variant="secondary"
            name="radio"
            value={radio.value}
            checked={radioValue === radio.value}
            onChange={(e) => setRadioValue(e.currentTarget.value)}
          >
            {radio.name}
          </ToggleButton>
        ))}
      </ButtonGroup>
      <br />
      <ToggleButton
        className="mb-2"
        id="toggle-check"
        type="checkbox"
        variant="outline-primary"
        checked={checked}
        value="1"
        onChange={(e) => setChecked(e.currentTarget.checked)}
      >
        Checked
      </ToggleButton>
      <br />
      <ButtonGroup>
        {radios.map((radio, idx) => (
          <ToggleButton
            key={idx}
            id={`radio-${idx}`}
            type="radio"
            variant={idx % 2 ? 'outline-success' : 'outline-danger'}
            name="radio"
            value={radio.value}
            checked={radioValue === radio.value}
            onChange={(e) => setRadioValue(e.currentTarget.value)}
          >
            {radio.name}
          </ToggleButton>
        ))}
      </ButtonGroup>
    </>
  );
}

export default ToggleButtonExample;
```

The above handles styling, But requires manually controlling the `checked` state for each radio or checkbox in the group.

For a nicer experience with checked state management use the `<ToggleButtonGroup>` instead of a `<ButtonGroup>` component. The group behaves as a form component, where the `value` is an array of the selected `value`s for a named checkbox group or the single toggled `value` in a similarly named radio group.

Uncontrolled

RESULT

Checkbox 1 (pre-checked) Checkbox 2 Checkbox 3 (pre-checked)
Radio 1 (pre-checked) Radio 2 Radio 3

LIVE EDITOR

```
import ToggleButton from 'react-bootstrap/ToggleButton';
import ToggleButtonGroup from 'react-bootstrap/ToggleButtonGroup';

function ToggleButtonGroupUncontrolled() {
  return (
    <>
      <ToggleButtonGroup type="checkbox" defaultValue={[1, 3]} className="mb-2">
        <ToggleButton id="tbg-check-1" value={1}>
          Checkbox 1 (pre-checked)
        </ToggleButton>
        <ToggleButton id="tbg-check-2" value={2}>
          Checkbox 2
        </ToggleButton>
        <ToggleButton id="tbg-check-3" value={3}>
          Checkbox 3 (pre-checked)
        </ToggleButton>
      </ToggleButtonGroup>
      <br />
      <ToggleButtonGroup type="radio" name="options" defaultValue={1}>
        <ToggleButton id="tbg-radio-1" value={1}>
          Radio 1 (pre-checked)
        </ToggleButton>
        <ToggleButton id="tbg-radio-2" value={2}>
          Radio 2
        </ToggleButton>
        <ToggleButton id="tbg-radio-3" value={3}>
          Radio 3
        </ToggleButton>
      </ToggleButtonGroup>
    </>
  );
}

export default ToggleButtonGroupUncontrolled;
```

Controlled

RESULT

Option 1 Option 2 Option 3

LIVE EDITOR

```
import { useState } from 'react';
import ToggleButton from 'react-bootstrap/ToggleButton';
import ToggleButtonGroup from 'react-bootstrap/ToggleButtonGroup';

function ToggleButtonGroupControlled() {
  const [value, setValue] = useState([1, 3]);

  /*
   * The second argument that will be passed to
   * `handleChange` from `ToggleButtonGroup`
   * is the SyntheticEvent object, but we are
   * not using it in this example so we will omit it.
   */
  const handleChange = (val) => setValue(val);

  return (
    <ToggleButtonGroup type="checkbox" value={value} onChange={handleChange}>
      <ToggleButton id="tbg-btn-1" value={1}>
        Option 1
      </ToggleButton>
      <ToggleButton id="tbg-btn-2" value={2}>
        Option 2
      </ToggleButton>
      <ToggleButton id="tbg-btn-3" value={3}>
        Option 3
      </ToggleButton>
    </ToggleButtonGroup>
  );
}

export default ToggleButtonGroupControlled;
```

API

Button

```
import Button from 'react-bootstrap/Button'
```

Name	Type	Default	Description
bsPrefix	`string | undefined`	`'btn'`	Change the underlying component CSS base class name and modifier class names prefix. **This is an escape hatch** for working with heavily customized bootstrap css.
active	`boolean | undefined`	`false`	Manually set the visual state of the button to `:active`

variant	`'primary' | 'secondary' | 'success' | 'danger' | 'warning' | 'info' | 'dark' | 'light' | 'link' | 'outline-primary' | 'outline-secondary' | 'outline-success' | 'outline-danger' | 'outline-warning' | 'outline-info' | 'outline-dark' | 'outline-light'`	`'primary'`	One or more button variant combinations
			buttons may be one of a variety of visual variants such as:
			`'primary', 'secondary', 'success', 'danger', 'warning', 'info', 'dark', 'light', 'link'`
			as well as "outline" versions (prefixed by 'outline-*')
			`'outline-primary', 'outline-secondary', 'outline-success', 'outline-danger', 'outline-warning', 'outline-info', 'outline-dark', 'outline-light'`

size	`'sm' | 'lg'`		Specifies a large or small button.
disabled	`boolean | undefined`	`false`	Disables the Button, preventing mouse events, even if the underlying component is an `<a>` element
href	`string | undefined`		Providing a `href` will render an `<a>` element, *styled* as a button.
onClick	`function | undefined`		Callback fired when the button is clicked.
type	`button | reset | submit | null`	`button`	Defines HTML button type attribute.

ToggleButtonGroup

```
import ToggleButtonGroup from 'react-bootstrap/ToggleButtonGroup'
```

Name	Type	Default	Description
name	`string | undefined`		An HTML `<input>` name for each child button. **Required if `type` is set to `'radio'`**
value	`any | undefined`		*controlled by: `onChange`, initial prop: `defaultValue`* The value, or array of values, of the active (pressed) buttons
onChange	`function | undefined`		*controls `value`* Callback fired when a button is pressed, depending on whether the type is 'radio' or 'checkbox', onChange will be called with the value or array of active values
type Required	`'checkbox' | 'radio'`		The input `type` of the rendered buttons, determines the toggle behavior of the buttons
size	`'sm' | 'lg' | undefined`		Sets the size for all Buttons in the group.
vertical	`boolean | undefined`		Make the set of Buttons appear vertically stacked.

ToggleButton

```
import ToggleButton from 'react-bootstrap/ToggleButton'
```

Name	Type	Default	Description
bsPrefix	`string | undefined`	`'btn-check'`	Change the underlying component CSS base class name and modifier class names prefix. **This is an escape hatch** for working with heavily customized bootstrap css.
type	`'checkbox' | 'radio' | undefined`		The `<input>` element `type`
name	`string | undefined`		The HTML input name, used to group like checkboxes or radio buttons together semantically
checked	`boolean | undefined`		The checked state of the input, managed by `<ToggleButtonGroup>` automatically
disabled	`boolean | undefined`		The disabled state of both the label and input
id Required	`string`		`id` is required for button clicks to toggle input.
onChange	`React.ChangeEventHandler<HTMLInputElement> | undefined`		A callback fired when the underlying input element changes. This is passed directly to the `<input>` so shares the same signature as a native `onChange` event.
value Required	`string | ReadonlyArray<string> | number`		The value of the input, should be unique amongst its siblings when nested in a `ToggleButtonGroup`.
inputRef	`React.Ref<HTMLInputElement> | undefined`		A ref attached to the `<input>` element

Previous
« Button group

Next
Cards »

Copyright © 2026 React Bootstrap. Built with Docusaurus.